ОТЧЁТ ПО ПРОЕКТУ

«Квантованные функции активации»

Проект P14

Командный проект, 2 человека

2026

1. Введение

Во время работы нейронной сети возникает много промежуточных чисел. К ним относятся веса модели, результаты умножений и значения функций активации. Обычно такие числа хранятся в формате `float32`, то есть для одного числа используется 32 бита. Это даёт высокую точность вычислений, но требует памяти.

Один из способов упростить модель — *квантование*. Простыми словами, это замена точного числа ближайшим разрешённым значением. Например, вместо $0{,}73$ можно использовать $0{,}67$ или 1. В данной работе исследуется, что произойдёт, если квантовать не только веса сети, но и значения функции активации.

Цель работы — изучить влияние квантования функций активации на точность, ошибку приближения, скорость и устойчивость нейронной сети.

Для достижения цели были поставлены задачи:

1) реализовать равномерное квантование выходных значений активации;
2) реализовать табличную LUT и кусочно-линейную PWL-аппроксимацию;
3) обучить базовую полносвязную сеть в формате `float32`;
4) сравнить квантование весов и активаций;
5) сравнить PTQ и QAT при одинаковых начальных весах и числе эпох;
6) проверить устойчивость к случайному шуму и FGSM-атаке;
7) оценить теоретическую память и время инференса.

2. Теоретические сведения

2.1. Нейронная сеть и функция активации

Полносвязный слой сначала вычисляет линейную комбинацию входов:

$$z^{(l)} = a^{(l-1)} W^{(l)} + b^{(l)}, \tag{1}$$

где $W^{(l)}$ — матрица весов, $b^{(l)}$ — смещение, а $a^{(l-1)}$ — выход предыдущего слоя. Затем применяется функция активации

$$a^{(l)} = \varphi\left(z^{(l)}\right). \tag{2}$$

В эксперименте использовалась функция

$$\tanh z = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}. \tag{3}$$

Она принимает значения от -1 до 1. Значения, полученные после применения tanh, называются *активациями*.

2.2. Биты, уровни и равномерное квантование

Бит может принимать значение 0 или 1. Поэтому с помощью b бит можно закодировать

$$L = 2^{b} \tag{4}$$

различных вариантов. Эти разрешённые варианты называются *уровнями квантования*. Например, при двух битах доступны четыре комбинации и, следовательно, четыре уровня.

Для равномерного квантования диапазона $[x_{\min}, x_{\max}]$ расстояние между соседними уровнями равно

$$\Delta = \frac{x_{\max} - x_{\min}}{L - 1}. \tag{5}$$

Исходное значение заменяется ближайшим уровнем:

$$Q_b(x) = x_{\min} + \Delta \cdot \operatorname{round}\left(\frac{\operatorname{clip}(x, x_{\min}, x_{\max}) - x_{\min}}{\Delta}\right). \tag{6}$$

Операция `clip` ограничивает число выбранным диапазоном, а `round` округляет номер уровня. Для tanh используется диапазон $[-1, 1]$. При двух битах уровни равны

$$-1, \quad -\frac{1}{3}, \quad \frac{1}{3}, \quad 1. \tag{7}$$

Например, значение $0{,}76$ заменяется на 1, а значение $0{,}2$ — на $1/3$.

2.3. LUT и PWL

Функцию активации можно упростить двумя дополнительными способами. В LUT (Look-Up Table) заранее выбираются входные узлы z_i и сохраняются точные значения $\tanh(z_i)$. При вычислении выбирается ближайший узел и возвращается значение из таблицы. В новом notebook параметром LUT является именно **число узлов**, а не число бит хранения. Использована таблица с нечётным числом узлов, поэтому точка $z = 0$ включена и выполняется $\tanh(0) = 0$.

PWL (Piecewise Linear) заменяет плавную кривую несколькими отрезками. Если $z \in [z_i, z_{i+1}]$, то используется линейная интерполяция

$$\widehat{\varphi}(z) = \varphi(z_i) + \frac{z - z_i}{z_{i+1} - z_i}\big(\varphi(z_{i+1}) - \varphi(z_i)\big). \tag{8}$$

LUT проще, но даёт ступенчатую функцию. PWL выполняет несколько арифметических операций, зато обычно точнее при одинаковом числе узлов.

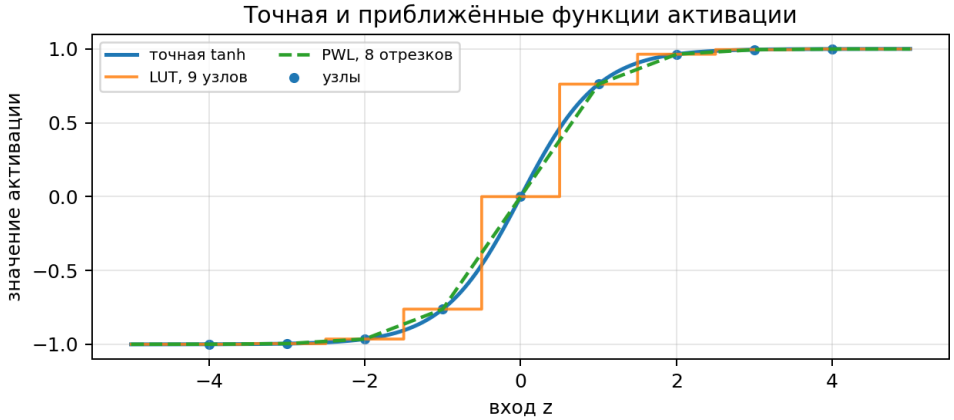


Рис. 1: Точная функция tanh, LUT с 9 узлами и PWL с 8 отрезками.

2.4. PTQ, QAT и STE

PTQ (Post-Training Quantization) означает, что сначала обучается обычная модель, а затем её числа квантуются без дополнительного обучения. QAT (Quantization-Aware Training) означает обучение с учётом квантования: при прямом проходе числа временно округляются, а точные копии весов сохраняются для постепенного обновления.

Операция округления имеет нулевой градиент почти везде. Поэтому при обучении применяется STE (Straight-Through Estimator): во время обратного прохода градиент проводят через квантователь приближённо, как через гладкую функцию. Такой же приём используется при проверке атаки на LUT.

3. Методика эксперимента

3.1. Данные и модель

Использован набор `digits` из библиотеки scikit-learn. Он содержит изображения рукописных цифр размером 8×8, поэтому на вход сети подаётся 64 признака. Значения пикселей приведены к диапазону $[0, 1]$ и сохранены в формате `float32`. Выборка разделена на 1347 обучающих и 450 тестовых объектов с сохранением долей классов. Зафиксировано случайное зерно `SEED=42`.

Архитектура модели:

$$64 \to 128 \to 64 \to 10. \tag{9}$$

На двух скрытых слоях применяется tanh, а выходной слой выдаёт десять логитов. Обучение выполнялось 60 эпох с размером мини-пакета 64 и оптимизатором Adam. Точность базовой модели составила

$$\mathrm{Acc}_{\mathrm{fp32}} = 0{,}9711. \tag{10}$$

3.2. Метрики

Основной метрикой является точность классификации. Для оценки приближения активации использовались среднеквадратичная и максимальная ошибки:

$$\mathrm{MSE} = \frac{1}{N}\sum_{i=1}^{N}\bigl(\varphi(z_i) - \widehat{\varphi}(z_i)\bigr)^2, \qquad E_{\max} = \max_i \bigl|\varphi(z_i) - \widehat{\varphi}(z_i)\bigr|. \tag{11}$$

Кроме этого измерялись доля изменившихся ответов, MSE логитов и средний запас логитов. Запас логитов равен разности между наибольшим и вторым по величине логитом. Чем он больше, тем увереннее ответ модели.

3.3. Модель угроз

Рассматривается white-box-сценарий: атакующему известны архитектура, веса и способ квантования. Для FGSM вход изменяется по формуле

$$x_{\mathrm{adv}} = \mathrm{clip}\left(x + \varepsilon\ \mathrm{sign}(\nabla_x \mathcal{L}), 0, 1\right), \tag{12}$$

где ε ограничивает изменение каждого пикселя. Также добавлялся гауссовский шум. Для каждого значения его стандартного отклонения результат усреднялся по 15 реализациям.

4. Результаты

4.1. Ошибка LUT и PWL

Ошибка вычислялась на сетке $z \in [-5, 5]$, а узлы аппроксимаций располагались на $[-4, 4]$. В таблице 1 приведены результаты для tanh.

Таблица 1: Ошибка аппроксимации функции tanh.

Узлы	LUT MSE	LUT $E_{\max}$	PWL MSE
3	$2{,}07 \cdot 10^{-1}$	0,964	$1{,}08 \cdot 10^{-1}$
5	$4{,}95 \cdot 10^{-2}$	0,761	$1{,}61 \cdot 10^{-2}$
9	$1{,}11 \cdot 10^{-2}$	0,460	$8{,}85 \cdot 10^{-4}$
17	$2{,}77 \cdot 10^{-3}$	0,243	$5{,}39 \cdot 10^{-5}$
33	$6{,}94 \cdot 10^{-4}$	0,122	$3{,}48 \cdot 10^{-6}$
65	$1{,}73 \cdot 10^{-4}$	0,0619	$2{,}51 \cdot 10^{-7}$

При увеличении числа узлов ошибка уменьшается. PWL заметно точнее ступенчатой LUT при одинаковом числе узлов, особенно при небольших таблицах. Это подтверждается и точностью модели: при 3 узлах LUT дала 0,100, а PWL с двумя отрезками — 0,962; при 9 узлах результаты составили 0,949 и 0,971 соответственно.

4.2. Квантование выходов активации

Таблица 2: PTQ выходов активации.

Биты	Точность	Изменённые ответы, %	MSE логитов	Запас логитов
1	0,960	2,89	1,9175	9,48
2	0,960	2,44	0,6058	7,92
3	0,973	0,67	0,1106	8,13
4	0,971	0,00	0,0250	7,99
6	0,971	0,00	0,0014	7,97
8	0,971	0,00	0,0001	7,96

Даже при одном и двух битах потеря точности относительно 0,9711 составила около одного процентного пункта. При 4–8 битах ни один итоговый класс не изменился, хотя логиты всё ещё немного отличались. Поэтому MSE логитов является более чувствительной диагностикой, чем одна точность.

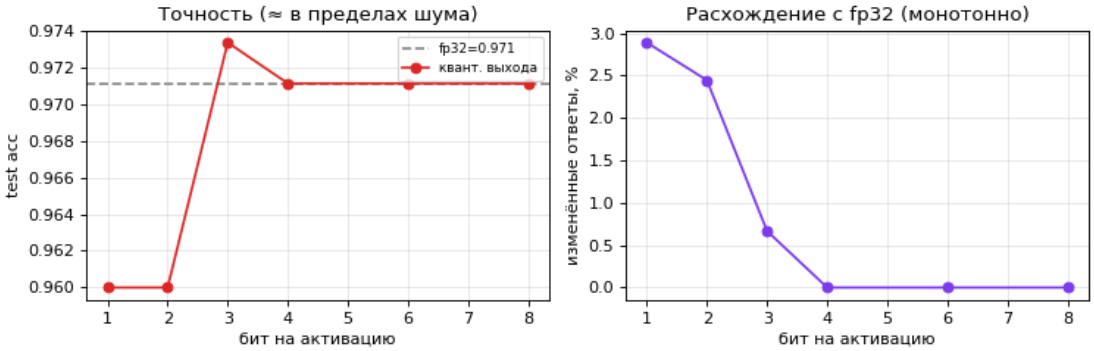


Рис. 2: Точность и доля изменившихся ответов при разной битности активаций.

4.3. Совместное квантование весов и активаций

В новом варианте веса и активации квантуются одним равномерным квантователем: при двух битах у обоих имеется четыре уровня. Поэтому их чувствительность можно сравнивать корректнее. Двухбитные активации дали точность 0,960, двухбитные веса — 0,927, а совместный режим — 0,922. Следовательно, в данной модели веса чувствительнее активаций.

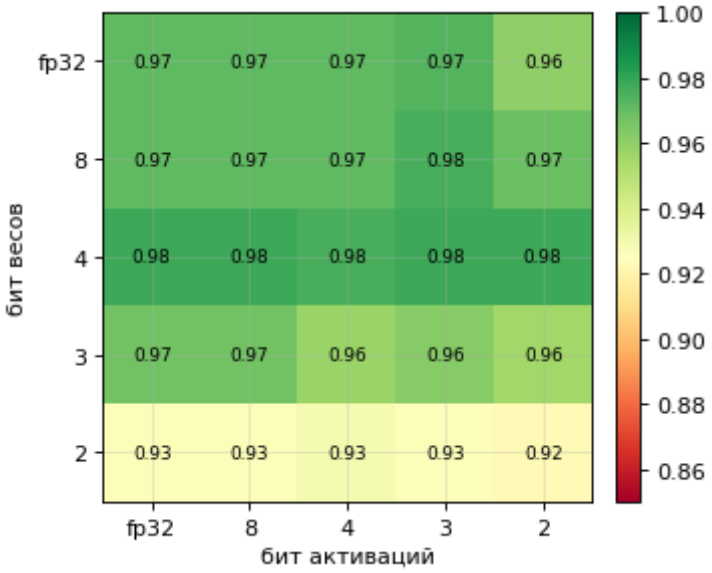


Рис. 3: Точность при сочетаниях битности весов и активаций.

4.4. PTQ и QAT

Для честного сравнения PTQ и QAT использованы одинаковые начальные веса и одинаковые 60 эпох. Результаты приведены в таблице 3.

Таблица 3: Сравнение PTQ и QAT.

Режим	PTQ	QAT	Изменение
Активации, 1 бит	0,960	0,964	$+0,004$
Активации, 2 бита	0,960	0,967	$+0,007$
Веса, 2 бита	0,927	0,980	$+0,053$
Веса и активации, 2 бита	0,922	0,969	$+0,047$

QAT мало влияет на мягкое квантование активаций, но заметно помогает при двухбитных весах. Модель во время обучения подстраивается под ошибку округления.

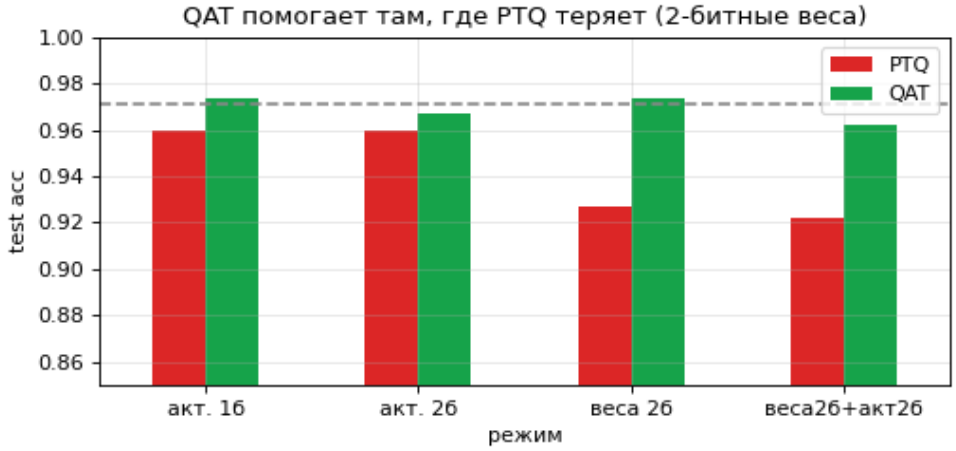


Рис. 4: Точность PTQ и QAT в четырёх режимах.

4.5. Устойчивость и маскировка градиента

FGSM быстро снижает точность всех моделей. При $\varepsilon = 0{,}1$ точности fp32, четырёхбитной и двухбитной моделей равны 0,431, 0,431 и 0,449. При $\varepsilon = 0{,}2$ они падают до 0,060, 0,053 и 0,040. Следовательно, квантование активаций не является защитой от

атаки.

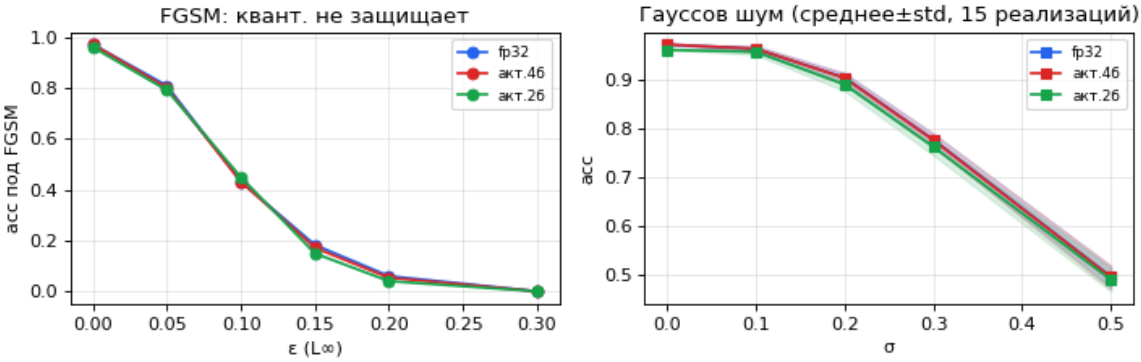


Рис. 5: FGSM и гауссовский шум. Для шума показано среднее и стандартное отклонение по 15 реализациям.

Для LUT отдельно проверена маскировка градиента. Если использовать истинный нулевой наклон ступенчатой функции, наивная атака сохраняет точность около 0,973 даже при $\varepsilon = 0{,}2$. Однако STE-атака при том же ε снижает точность до 0,062. Поэтому высокая устойчивость наивной проверки является ложной и связана с неправильным градиентом.

4.6. Память и скорость

Теоретическая доля памяти для одной b-битной активации относительно fp32 равна

$$r_{\text{mem}} = \frac{b}{32} \cdot 100\%. \tag{13}$$

Таблица 4: Теоретическая память и точность PTQ активаций.

Биты	Память от fp32	Точность
32	100%	0,971
8	25%	0,971
4	12,5%	0,971
2	6,25%	0,960
1	3,13%	0,960

В сохранённом запуске время обработки 9000 объектов составило около 48,3 мс для точной tanh, 59,2 мс для квантования выхода, 63,5 мс для LUT с 65 узлами и 111,5 мс для PWL с 32 отрезками. Эти значения зависят от компьютера и загрузки среды. В простой NumPy-реализации устойчивого ускорения не обнаружено. Теоретическая экономия памяти не означает автоматического ускорения: оно требует библиотек и оборудования с эффективной поддержкой низкой битности.

5. Ограничения работы

Результаты относятся к конкретному учебному эксперименту. Использован небольшой набор изображений 8×8, одна полносвязная архитектура и только функция tanh. Обучение выполнено при одном случайном зерне. Шум усреднялся по 15 реализациям, но FGSM запускался один раз для каждого ε. Из атак рассмотрен только FGSM; PGD могла бы дать более строгую оценку. Память оценивалась теоретически, а скорость — в NumPy без специального целочисленного оборудования.

6. Заключение

В работе были реализованы равномерное квантование выхода активации, LUT и PWL. Основные результаты следующие:

1) активации tanh можно квантовать до одного-двух бит с небольшой потерей точности;
2) при одинаковом числе узлов PWL значительно точнее ступенчатой LUT;
3) при одинаковом четырёхуровневом квантователе двухбитные веса чувствительнее двухбитных активаций;
4) QAT почти не нужен для мягкого квантования активаций, но восстанавливает точность двухбитных весов;
5) квантование активаций не защищает от FGSM;
6) высокая устойчивость ступенчатой LUT при наивной атаке является маскировкой градиента;
7) низкая битность даёт теоретическую экономию памяти, но в NumPy не гарантирует ускорение.

Таким образом, квантование функций активации может уменьшить разрядность промежуточных данных без существенной потери качества. Однако перед практическим применением необходимо отдельно проверять точность, скорость и устойчивость на выбранной модели и оборудовании.

Список литературы

[1] I. Hubara, M. Courbariaux, D. Soudry, R. El-Yaniv, Y. Bengio. *Quantized Neural Networks: Training Neural Networks with Low Precision Weights and Activations*. arXiv:1609.07061, 2016.

[2] B. Jacob, S. Kligys, B. Chen et al. *Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference*. CVPR, 2018.

[3] I. Goodfellow, J. Shlens, C. Szegedy. *Explaining and Harnessing Adversarial Examples*. ICLR, 2015.